

Number crunching.

ip: n = 1 2 3 4
op: 4 3 2 1

To get last digit $\rightarrow \%$
To remove last digit $\rightarrow /$

4 3 2 1

while (n != 0) (or) while (n > 0)

Without $\times 10 \rightarrow$ for add

$$4 + 3 = 7.$$

With $\times 10 \rightarrow$ for reverse

$$40 + 3 = 43$$

$$n \% 10 \rightarrow 4$$

$$n / 10 \rightarrow 123$$

$$n \% 10 \rightarrow 3$$

$$n / 10 \rightarrow 12$$

$$n \% 10 \rightarrow 2$$

$$n / 10 \rightarrow 1$$

$$n \% 10 \rightarrow 1$$

$$n / 10 \rightarrow 0$$

```
import java.util.*;
public class Main{
    public static int ReverseDigit(int n){
        int rev = 0;
        while(n!=0){
            int ld = n%10;
            rev = rev * 10 + ld;
            n /= 10;
        }
        return rev;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        System.out.println(ReverseDigit(n));
        // (or)
        // int ans = ReverseDigit(n);
        // System.out.println(ans);
    }
}
```

Sum of Digit.

ip: 1 2 3 4
op: 10

```
public static int SumDigit(int n){
    int sum = 0;
    while(n!=0){
        int ld = n%10;
        sum = sum + ld;
        n /= 10;
    }
    return sum;
}
```

count digit

ip: 1 2 3 4
op: 4

```
public static int countDigits(int n){
    int cnt = 0;
    while(n!=0){
        cnt++;
        n /= 10;
    }
    return cnt;
}
```

Swap 1st & last Digit.

ip: 24857

op: 74852

```
public static int swap1stAndLast(int n){
    int ld = n%10;
    n = n/10;
    n = ReverseDigit(n);
    int fd = n%10;
    n = n/10;
    n = n*10 + ld;
    n = ReverseDigit(n);
    n = n*10 + fd;
    return n;
}
```

24857

ld = n%10; //7

n = n/10; // 2485

n = rev(n); // 5842

fd = n%10; //2

n = n/10; // 584

n = n*10 + ld; // 5847

n = rev(n); // 7485

n = n*10 + fd; // 74852

return n;

Reverse Digit Without input destruction.

ip: 1234

op: 4321

Note:

Without using temp or 2nd variable to store the input.

div = 1;

1234 → 1234 % 10 → 4 / 1 = 4

1234 → 1234 % 100 → 34 / 10 = 3

1234 → 1234 % 1000 → 234 / 100 = 2

1234 → 1234 % 10000 → 1234 / 1000 = 1

```
public static int ReverseWithoutDesturction(int n){
    int rev = 0;
    int div = 1;
    while(div <= n){
        int ed = (n%(div*10))/div;
        rev = rev * 10 + ed;
        div = div * 10;
    }
    System.out.println(n);
    return rev;
}
```

div = 1000

n = 1234

ed = (n % (div*10)) / div;

while (div <= n){
div = div*10;

Compute x pow n

ip: base = 2 pow = 5

op: 32.

$$2 \times 2 \times 2 \times 2 \times 2 = 32.$$

```
public static int ComputeXpowN(int base, int power) {  
    int total = 1;  
    for(int i=1; i<=power; i++) {  
        total = total * base; //2  
    }  
    return total;  
}
```

Note: Whenever we know the range use **for loop**.
Whenever we don't know the range use **while loop**.

Armstrong Number

ip: 153.

cnt = 3.

$$= 1^3 + 5^3 + 3^3$$

$$= 1 + 125 + 27$$

$$= 153.$$

op: True.

st1: Count No of Digit.

st2: power each digit with No of digit.

st3: Sum of power value.

st4: Check Sum & Input are same → return true.

ip: 1634.

cnt = 4

$$= 1^4 + 6^4 + 3^4 + 4^4$$

$$= 1 + 1296 + 81 + 256.$$

$$= 1634$$

op: True.

```
public static int countDigits(int n) {  
    int cnt = 0;  
    while(n!=0) {  
        cnt++;  
        n /= 10;  
    }  
    return cnt;  
}  
  
public static int ComputeXpowN(int base, int power) {  
    int total = 1;  
    for(int i=1; i<=power; i++) {  
        total = total * base; //2  
    }  
    return total;  
}  
  
public static boolean ArmstrongNumber(int n){  
    int NoOfDigit = countDigits(n); //4  
    int div = 1;  
    int sum = 0;  
    while(div<=n){  
        int ld = (n%(div*10))/div; //  
        int power = ComputeXpowN(ld, NoOfDigit);  
        sum += power;  
        div = div * 10;  
    }  
    /*  
    if(n==sum) return true;  
    return false;  
    */  
    return n==sum;  
}
```

1281. Subtract the Product and Sum of Digits of an Integer

Input: n = 234

Output: 15

Explanation:

Product of digits = $2 * 3 * 4 = 24$

Sum of digits = $2 + 3 + 4 = 9$

Result = $24 - 9 = 15$

```
public int subtractProductAndSum(int n) {
    int prod = 1;
    int sum = 0;
    int div = 1;
    while(div <= n){
        int ed = (n%(div*10))/div;
        prod *= ed;
        sum += ed;
        div *= 10;
    }
    return prod - sum;
}
```

2119. A Number After a Double Reversal

Input: num = 526

Output: true

Explanation: Reverse num to get 625, then reverse 625 to get 526, which equals num.

```
class Solution {
    public boolean isSameAfterReversals(int num) {
        if(num==0) return true; //0
        if(num%10==0) return false;
        return true;
    }
}
```

$n = 526$

1st rev = 625

2nd rev = 526 ✓

$n = 1800$

1st rev = 0081 → 81

2nd rev = 18 ✗

$n = 1860$

1st rev = 681

2nd rev = 186 ✗

292. Nim Game

You are playing the following Nim Game with your friend:

- Initially, there is a heap of stones on the table.
- You and your friend will alternate taking turns, and **you go first**.
- On each turn, the person whose turn it is will remove 1 to 3 stones from the heap.
- The one who removes the last stone is the winner.

Given `n`, the number of stones in the heap, return `true` if you can win the game assuming both you and your friend play optimally, otherwise return `false`.

Input: n = 4

Output: false

Explanation: These are the possible outcomes:

- You remove 1 stone. Your friend removes 3 stones, including the last stone. Your friend wins.
 - You remove 2 stones. Your friend removes 2 stones, including the last stone. Your friend wins.
 - You remove 3 stones. Your friend removes the last stone. Your friend wins.
- In all outcomes, your friend wins.

$n = 4$

me - 1, 2, 3

friend - 3, 2, 1

$n = 5$

me - 1, 1, 1.

friend - 3, 2, 1

me - 1, 2, 3

$n = 8$

me - 3, 2

friend - 1, 2

me - (1, 2, 3) - (3, 2, 1)

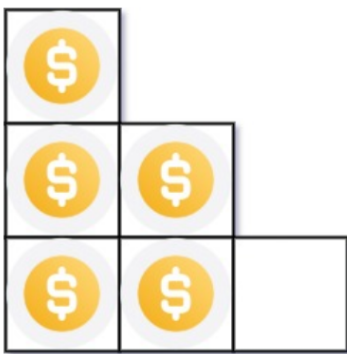
friend - (3, 2, 1) - (1, 2, 3)

```
class Solution {  
    public boolean canWinNim(int n) {  
        return n%4!=0;  
    }  
}
```

441. Arranging Coins

You have n coins and you want to build a staircase with these coins. The staircase consists of k rows where the i^{th} row has exactly i coins. The last row of the staircase **may be** incomplete.

Given the integer n , return the number of **complete rows** of the staircase you will build.



$n \rightarrow$ No of coins.

Step = 1

$n = n - \text{step}; // 4$

Step = 2.

$n = n - \text{step}; // 2$

Step = 3 $\rightarrow n = 2$ x incomplete.

Input: $n = 5$

Output: 2

Explanation: Because the 3rd row is incomplete, we return 2.

```
public int arrangeCoins(int n) {  
    int step = 1;  
    int cnt = 0;  
    while(n >= step){  
        cnt++;  
        n = n - step;  
        step++;  
    }  
    return cnt;  
}
```

2520. Count the Digits That Divide a Number

Input: num = 121

Output: 2

Explanation: 121 is divisible by 1, but not 2. Since 1 occurs twice as a digit, we return 2.

```
class Solution {
    public int countDigits(int num) {
        int cnt = 0;
        int div = 1;
        while (div <= num) {
            int ed = (num % (div * 10)) / div;
            if (num % ed == 0) {
                cnt++;
            }
            div *= 10;
        }
        return cnt;
    }
}
```

2652. Sum Multiples

Input: n = 7

Output: 21

Explanation: Numbers in the range [1, 7] that are divisible by 3, 5, or 7 are 3, 5, 6, 7. The sum of these numbers is 21.

```
class Solution {
    public int sumOfMultiples(int n) {
        int sum = 0;
        for(int i=1;i<=n;i++){
            if(i%3==0 || i%5==0 || i%7==0){
                sum += i;
            }
        }
        return sum;
    }
}
```

2160. Minimum Sum of Four Digit Number After Splitting Digits

You are given a **positive** integer num consisting of exactly four digits. Split num into two new integers new1 and new2 by using the **digits** found in num. **Leading zeros** are allowed in new1 and new2, and **all** the digits found in num must be used.

- For example, given num = 2932, you have the following digits: two 2's, one 9 and one 3. Some of the possible pairs [new1, new2] are [22, 93], [23, 92], [223, 9] and [2, 329].

Input: num = 2932

Output: 52

Explanation: Some possible pairs [new1, new2] are [29, 23], [223, 9], etc. The minimum sum can be obtained by the pair [29, 23]: 29 + 23 = 52.

num = 2 9 3 2

split $\rightarrow [1, 3], [3, 1]$ ✗

split $\rightarrow [2, 2]$ ✓

① $\rightarrow 29 + 32 = 61$ ✗

② $\rightarrow 29 + 23 = 52$ ✓

$\text{min}_1 = 9, \text{min}_2 = 9$

$\text{max}_1 = 0, \text{max}_2 = 0$

$(\text{min}_1 \times 10 + \text{max}_1) + (\text{min}_2 \times 10 + \text{max}_2)$